Decision Trees

O'nel Hounnouvi

1 Modélisation

On dispose d'un jeu de données (X, y) avec $X \in \mathbb{R}^{n \times d}$ (n échantillons, d features/variables) et $y \in \mathbb{R}^n$ la cible (continue en régression, labels discrets en classification).

Un arbre de décision modélise une fonction constante par morceaux. Il divise l'espace des variables $\mathcal{X} \subseteq \mathbb{R}^d$ en M sous-régions $R_1, R_2, \ldots, R_M$ formant une partition. Pour une observation x, la prédiction de l'arbre $f(x)$ est déterminée par la région R_m dans laquelle elle tombe :

$$f(x) = \sum_{m=1}^{M} c_m \cdot \mathbb{1}_{\{x \in R_m\}}$$

où c_m est la valeur prédite (la constante) associée à la feuille m.

Le modèle est un arbre binaire : chaque nœud interne est défini par un couple (j, t) , un indice de variable et un seuil, et chaque feuille porte une valeur de prédiction. Pour un exemple x, la prédiction consiste à descendre l'arbre : au nœud (j, t), on va à gauche si $x_j \leq t$, à droite sinon, jusqu'à atteindre une feuille.

Remarque : Une feuille m est atteinte par un chemin de longueur p depuis la racine, qui impose des conditions $C_1, \ldots, C_p$ le long de la descente, donc

$$R_m = \bigcap_{k=1}^{p} C_k$$

Chaque condition C_k est de la forme $\{x : x_{j_k} \leq t_k\}$ ou $\{x : x_{j_k} > t_k\}$ et ne porte que sur une seule variable j_k. En regroupant les conditions par variable, l'intersection de toutes celles portant sur x_j est une intersection d'intervalles de $\mathbb{R}$, donc un intervalle $]a_{m,j}, b_{m,j}]$ (avec $a_{m,j} = -\infty$ ou $b_{m,j} = +\infty$). D'où

$$R_m = \bigcap_{j=1}^{d} \{x : x_j \in]a_{m,j}, b_{m,j}]\} = \prod_{j=1}^{d}]a_{m,j}, b_{m,j}]$$

qui est un pavé (hyperrectangle), éventuellement non borné.

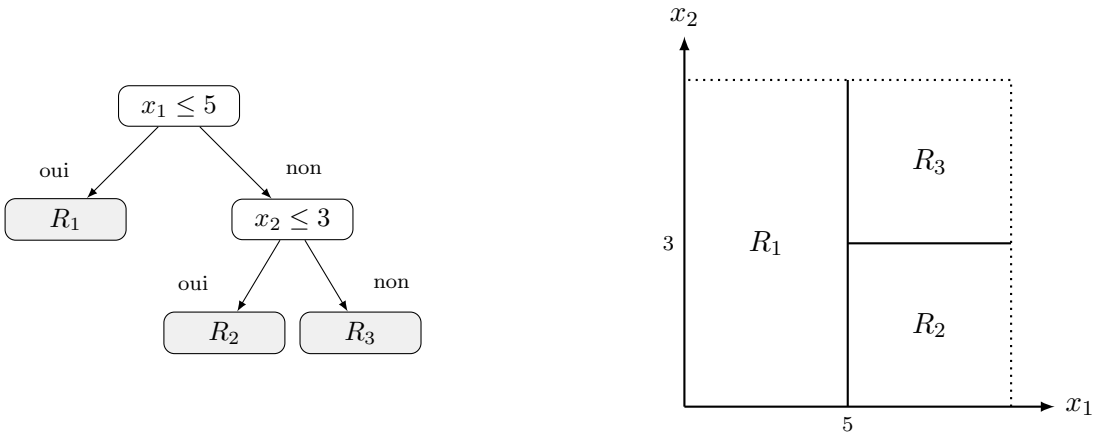


Figure 1: Un arbre à deux splits (gauche) et la partition de $\mathbb{R}^2$ en trois rectangles qu'il induit (droite). Chaque feuille correspond à une région R_m ; les traits pleins internes sont les frontières entre régions, le cadre en pointillés n'est qu'une fenêtre de visualisation. Aucune des trois régions n'est réellement bornée.

2 Objectif

On veut trouver la partition $\{R_m\}$, et les valeurs des feuilles $\{c_m\}$ qui minimisent l'erreur empirique

$$\min_{M,\ \{R_m\},\ \{c_m\}} \sum_{m=1}^{M} \sum_{x_i \in R_m} L(y_i, c_m)$$

où L est une perte adaptée à la tâche : la perte 0/1, $L(y, c) = \mathbf{1}_{\{y \neq c\}}$, en classification ; l'erreur quadratique, $L(y, c) = (y - c)^2$, en régression.

Valeur optimale d'une feuille, à partition fixée. Pour une région R_m donnée, le sous-problème sur c_m se résout en fermé :

$$c_m^\star = \arg\min_c \sum_{x_i \in R_m} L(y_i, c)$$

où L est une fonction de perte adaptée à la tâche

Cas régression. Soit $N_m = |R_m|$ le nombre d'échantillons dans la feuille et $J(c) = \sum_{x_i \in R_m} (y_i - c)^2$ la perte quadratique totale sur cette feuille, en fonction du paramètre c. On a :

$$\frac{dJ(c)}{dc} = \sum_{x_i \in R_m} -2(y_i - c) = 0$$

$$\sum_{x_i \in R_m} y_i \ - \sum_{x_i \in R_m} c = 0$$

$$\sum_{x_i \in R_m} y_i = N_m \cdot c$$

$$c_m^\star = \frac{1}{N_m} \sum_{x_i \in R_m} y_i = \bar{y}_{R_m}$$

($d^2J/dc^2 = 2N_m > 0$, J est donc convexe en c et le point critique est bien un minimum)

Cas classification. La fonction de perte $J(c) = \sum_{x_i \in R_m} \mathbf{1}_{\{y_i \neq c\}}$ compte le nombre de classifications erronées. Comme c ne prend qu'un nombre fini de valeurs discrètes (les classes), pas de dérivée possible.

Soit $N_m(c) = |\{x_i \in R_m : y_i = c\}|$ le nombre d'échantillons de R_m qui portent la classe c. En comptant les bien classés plutôt que les mal classés :

$$J(c) = \sum_{x_i \in R_m} \mathbf{1}_{\{y_i \neq c\}} = \sum_{x_i \in R_m} \big(1 - \mathbf{1}_{\{y_i = c\}}\big) = N_m - N_m(c)$$

N_m ne dépend pas de c (c'est la taille de la feuille), donc minimiser $J(c) = N_m - N_m(c)$ sur c revient à maximiser $N_m(c)$:

$$c_m^\star = \arg\min_c \big[N_m - N_m(c)\big] = \arg\max_c N_m(c)$$

c'est-à-dire la classe dont l'effectif $N_m(c)$ est le plus grand dans R_m : la classe majoritaire.

En résumé, une feuille renvoie une moyenne en régression et un mode en classification.

Le problème global est intraitable. Trouver conjointement M, $\{R_m\}$ et $\{c_m\}$ qui minimisent l'erreur totale est un problème d'optimisation combinatoire NP-difficile.

L'algorithme CART (Classification And Regression Trees) contourne ce problème en construisant l'arbre avec une approche gloutonne : à chaque noeud, on choisit le split qui maximise localement une baisse d'impureté, mesurée par un critère $I(y)$ calculable sur un ensemble de labels/cibles. C'est cette mesure qu'on formalise maintenant.

Régression : variance (MSE).

$$I_{\text{MSE}}(y) = \frac{1}{|y|} \sum_{i=1}^{|y|} (y_i - \bar{y})^2 = \text{Var}(y)$$

En régression, l'impureté d'un nœud mesure la dispersion des valeurs cibles y_i autour de la valeur constante prédite pour ce nœud. En injectant la constante optimale dérivée précédemment ($\hat{c} = \bar{y}$, la moyenne empirique) dans la fonction de perte quadratique, on obtient bien l'impureté $I_{\text{MSE}}(y)$ qui est la variance.

Classification : impureté de Gini.

$$I_{\text{Gini}}(y) = 1 - \sum_{k=1}^{K} p_k^2 \quad \in \left[0, 1 - \frac{1}{K}\right], \qquad p_k = \frac{|\{i : y_i = k\}|}{|y|}$$

où 0 indique une pureté parfaite (toutes les observations appartiennent à une seule classe) et $1 - \frac{1}{K}$ indique une impureté maximale (les observations sont réparties de manière équiprobable entre les K classes).

Pourquoi cette formule et pas une autre : $I_{\text{Gini}}(y)$ est la probabilité de mal classer un exemple si on lui attribue un label k tiré aléatoirement selon la distribution empirique $p_1, \ldots, p_K$. En effet, pour une observation de label réel k (qui arrive avec probabilité p_k), la probabilité qu'elle soit mal classée est $1 - p_k$. Comme les événements "appartenir à la classe k" sont disjoints, l'erreur totale attendue pour toutes les classes K est donc la somme :

$$I_{\text{Gini}}(y) = \sum_{k=1}^{K} \underbrace{p_k}_{\text{vraie classe}=k} \cdot \underbrace{(1 - p_k)}_{\text{deviner}\neq k} = \sum_{k=1}^{K} p_k - \sum_{k=1}^{K} p_k^2 = 1 - \sum_{k=1}^{K} p_k^2$$

3 Dérivation

Pour un split candidat (j, t) qui partitionne y en y_L (gauche, $x_j \leq t$) et y_R (droite, $x_j > t$), on définit le gain d'information IG par :

$$IG(y, y_L, y_R) = I(y) - W_L\, I(y_L) - W_R\, I(y_R)$$

avec $W_L = |y_L|/|y|$, $W_R = |y_R|/|y|$ et I la fonction d'impureté.

C'est la baisse d'impureté espérée : l'impureté du parent moins la moyenne pondérée des impuretés des deux enfants.

Recherche du meilleur split. Pour chaque feature $j \in \{1, \ldots, d\}$, on teste les seuils $t \in \{\text{valeurs distinctes de } X_{\cdot,j}\}$ pour couvrir toutes les partitions atteignables. La coupure optimale (j^*, t^*) est celle qui maximise le gain d'information IG. On cherche alors :

$$(j^\star, t^\star) = \arg\max_{j,\,t}\ IG\big(y, y_L(j,t), y_R(j,t)\big)$$

4 Algorithme

Algorithme de construction. La construction de l'arbre suit une logique récursive descendante (top-down). L'algorithme prend en entrée un ensemble d'observations S, évalue le critère d'arrêt, et, s'il n'est pas atteint, cherche la coupure optimale avant de s'appeler récursivement sur les partitions créées.

Algorithm 1 _build_tree$(X, y, \text{profondeur})$

Fonction `_build_tree`$(X, y, \textit{profondeur})$**:**
- **if** $|\{y_i\}| \leq 1$ *ou* $\textit{profondeur} = \textit{profondeur}_{\max}$ **then**
  - **return** Feuille(`_calculate_leaf_value`(y))
- $(IG^\star, j^\star, t^\star) \leftarrow$ `_get_best_split`(X, y)
- **if** $IG^\star \leq 0$ **then**
  - **return** Feuille(`_calculate_leaf_value`(y))
- $(X_L, y_L), (X_R, y_R) \leftarrow$ `_split`$(X, y, j^\star, t^\star)$
- gauche $\leftarrow$ `_build_tree`$(X_L, y_L, \textit{profondeur} + 1)$
- droite $\leftarrow$ `_build_tree`$(X_R, y_R, \textit{profondeur} + 1)$
- **return** Nœud$(j^\star, t^\star, \text{gauche}, \text{droite})$

5 Lien code

- $I_{\text{Gini}}(y) \rightarrow$ `DecisionTreeClassifier._calculate_impurity`
- $I_{\text{MSE}}(y) \rightarrow$ `DecisionTreeRegressor._calculate_impurity`
- $IG(y, y_L, y_R) \rightarrow$ `BaseDecisionTree._information_gain`
- $(j^\star, t^\star) = \arg\max IG \rightarrow$ `BaseDecisionTree._get_best_split`
- Valeur optimale d'une feuille $\rightarrow$ `BaseDecisionTree._calculate_leaf_value`
- construction récursive $\rightarrow$ `BaseDecisionTree._build_tree`
- descente de l'arbre pour prédire $\rightarrow$ `_traverse_tree` / `predict`

6 Complexité

Dans `_get_best_split`, pour une feature donnée il y a jusqu'à n seuils candidats (`np.unique`), et pour chaque seuil on recalcule l'impureté des deux côtés en $O(n)$. Ça donne $O(n^2)$ par feature, donc $O(n^2 d)$ par nœud — c'est le coût réel de l'implémentation actuelle (naïve, sans tri préalable ni cumul incrémental des comptes).

Piste d'amélioration : une version optimisée trierait chaque feature une fois ($O(n \log n)$) et mettrait à jour les comptes par classe de façon incrémentale en balayant les seuils triés, ramenant le coût à $O(nd \log n)$ par nœud.

En comptant tous les nœuds d'un niveau de l'arbre (au total n échantillons par niveau), et avec une profondeur bornée par `max_depth`, le coût total naïf est en $O(\text{max_depth} \cdot n^2 \cdot d)$.